Fewer Tokens, Fewer Silent Errors: Output Format Design for Small Language Models in Real Time Order Parsing

Umar Ahmed
ITMO University
ORCID: 0009-0003-6040-6328
uakhmed@itmo.ru

Maxim Duanev
ITMO University
ORCID: 0009-0009-2988-9839
mduanev@itmo.ru

Mohammad Nour Al Awad
ITMO University
ORCID: 0009-0008-5078-9800
MohammadNourAlAwad@itmo.ru

Sergey Ivanov
ITMO University
ORCID: 0000-0002-1128-2942
svivanov@itmo.ru

Ivan V. Khodnenko
ITMO University
ORCID: 0009-0003-7787-7126
ivan.khodnenko@itmo.ru

***Abstract*: Small language models are cheap to fine tune and fast to serve, but they hallucinate, and a hallucination that parses cleanly is invisible to the system that consumes it. We ask whether the *output format* can be designed to reduce that risk on a narrow task. We fine tune Qwen3.5-0.8B with LoRA on the public PIZZA semantic parsing benchmark using six lossless encodings of the same target tree, arranged as a ladder from 45.9 down to 11.3 tokens per utterance, with two matched token pairs that hold length fixed while varying surface form. Across 90 runs we find that shortening the output does *not* by itself improve accuracy: at 1000 training examples every ten tokens removed costs about 1.5 points of exact match. What matters is whether the surface form is made of natural words. A word based encoding matches the native s-expression on accuracy (0.878 versus 0.869, inside a measured rerun noise floor of 0.032) while using 4.0× fewer tokens, running 3.7× faster at batch size one, producing 2.5× fewer hallucinated entities, and halving the rate of errors that parse cleanly but are wrong. We argue this last quantity, which we call the *silent error rate*, is the right objective for deploying small models. The matched-pair result replicates on LFM2.5-350M, whose vocabulary is four times smaller, where the word based encoding costs *more* tokens than its symbolic twin and is still the more accurate of the two. We report a café ordering system built on the finding that runs at 80 ms per order on a consumer laptop.**

***Index Terms*: semantic parsing, small language models, hallucination, LoRA, output formats**

I. Introduction

A small language model is attractive for a narrow task. It fine tunes in minutes, runs on a laptop, and costs almost nothing per call. The objection is reliability: small models hallucinate [8], and in a structured output setting a hallucination is not an obviously broken string. It is a well formed tree describing an order nobody placed.

This distinction matters more than overall accuracy. An output that fails to parse is a *detectable* failure: the caller sees it, rejects it, and retries. An output that parses but is wrong is a *silent* failure: the caller has no signal, and a customer receives the wrong order. We therefore separate the two and treat the silent rate as the quantity to minimise.

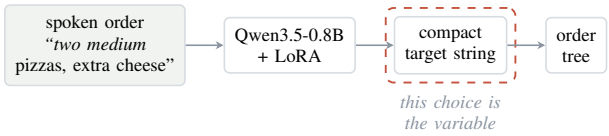


Fig. 1: The pipeline. Everything is held fixed except the format the model is trained to emit.

Our question is whether the target format itself is a lever. The same semantic tree can be written many ways, and the choice changes how many tokens the model must emit, how fast it responds, and apparently how often it invents content. Our initial hypothesis was simple: fewer output tokens should mean less room to hallucinate, faster responses, and cheaper fine tuning.

That hypothesis is not supported in the form we posed it. Shortening the output alone does not improve accuracy, and at low data it clearly hurts. But a sharper version survives: shortening helps when the surface form stays close to natural language, and hurts when it becomes symbolic. Section VI shows this with matched token controls that separate length from form.

Contributions. (1) A ladder of six lossless encodings of the PIZZA target tree spanning 45.9 to 11.3 tokens, with matched token pairs that isolate length from surface form. (2) Evidence that length alone is not the mechanism, from 15 paired comparisons at equal token budget, all 15 of which separate in the same direction. (3) The silent error rate as a deployment objective, with a word based encoding halving it relative to the native format. (4) A measured rerun noise floor of 0.032 exact match, which bounds what any three seed comparison on this task can claim.

II. Related Work

a) Target representation choice.: That the choice of meaning representation affects a model's accuracy is not new. Dong and Lapata [2] already found that decoding logical forms as trees rather than flat sequences changed results, and Shin *et al.* [3] made the point closest to ours: prompting a language model to produce a *canonical natural-language form*, which a grammar then maps to the meaning representation, beats prompting it for the meaning representation directly. Our finding is the same phenomenon in a different regime, and we try to pin down its mechanism. Shin *et al.* work few-shot, with a frozen model and constrained decoding, where one plausible explanation is simply that natural-looking strings sit closer to the pretraining distribution a frozen model can only be steered towards. We fine tune, so the model is given the chance to learn whatever notation we ask for, and the effect persists. We also hold token count fixed across a matched pair, which separates naturalness from length, and we replicate across two subword vocabularies.

b) Does imposing a format cost accuracy?: Tam *et al.* [4] ask a question adjacent to ours and reach a compatible answer: restricting a model to JSON, especially through constrained decoding, degrades performance relative to free-form generation, and not mainly through parse failures. Read together with our results, the lesson is not that structure is harmful but that *notation* is the cost. We never relax the requirement that output be machine parseable; we vary only how it is spelled, and recover much of the gap at no cost in parseability.

c) Constrained decoding.: A large body of work makes invalid output impossible rather than unlikely, by masking the token distribution against a grammar or automaton: PICARD [5] for SQL, grammar-constrained decoding for structured NLP tasks generally [6], and efficient vocabulary indexing to make the mask cheap [7]. This is complementary to what we study and partly orthogonal to it. Constrained decoding can drive parse validity to one, which converts our detectable failures into something else, but it cannot make a well-formed tree correct. The silent error rate we measure is exactly the part of the problem constrained decoding does not address, and the format comparison remains live underneath it: we ship a grammar for the task and report the unconstrained numbers, because an arm's unconstrained validity is itself diagnostic of how well the model has internalised the notation.

d) Hallucination.: Surveys of hallucination in generation [8] largely treat it as a faithfulness problem in open-ended text, where the judgement of what is invented is itself a modelling problem. A closed-schema parsing task makes the question decidable: an entity in the output that is absent from the input and from the schema is invented, and can be counted. We use that to report hallucination as a rate per utterance rather than as a human judgement.

e) Tokenization.: Subword segmentation [10] determines how many tokens a string costs, and vocabulary fit is known to move downstream accuracy, not just sequence length [11]. Our tokenizer-floor measurement is a narrow instance: shortening a string stops shortening its token sequence once the string leaves the vocabulary, which is why our ladder has a floor at about ten tokens, and why the same format costs different amounts on two different vocabularies.

III. Task and System

A. The café parser

The work began with a deployed system. A café ordering booth must turn a spoken order, already transcribed, into a structured ticket. Orders arrive in Russian and English, with diminutives, self corrections and off menu items. We fine tuned Qwen3.5-0.8B with LoRA on a synthetic generator covering ten drinks, three sizes, five milks and three syrups, with a compact target format:

```
cap L oat;lat x2
```

where each item is `drink [size] [milk] [syrup] [xN]` and `?` marks an utterance that is not an order. On a hand written 49 case evaluation set, written separately from the generator and containing slang the generator never produces, the model reaches 48 of 49, emitting about three output tokens per order with a median latency of 80 ms. That figure is measured on an Apple M1 through a local Ollama server using the same Q8_0 GGUF, which is 795 MiB on disk. The compact format uses roughly three times fewer output tokens than an equivalent JSON encoding, which is most of that latency.

This system works, but it cannot support a claim. The evaluation set is small, and the same author wrote both the generator and the test cases. To say anything general we moved to a public benchmark.

B. PIZZA

PIZZA [1] is a semantic parsing dataset of spoken pizza and drink orders paired with executable target trees. We use the full published training pool of 2,456,446 utterances, the 348 utterance development split and the 1357 utterance test split, of which 434 are marked as failures of the authors' PCFG baseline. Our largest training size is 2,456,443: the pool less 3 rows whose source and target are identical to an evaluation utterance. Those rows are dropped from training rather than from the evaluation splits, so the splits remain exactly as published. Targets look like:

```
(ORDER (PIZZAORDER (NUMBER 2 )
(SIZE MEDIUM ) (COMPLEX_TOPPING
(QUANTITY EXTRA ) (TOPPING CHEESE ) ) ) )
```

We verified that no development or test utterance appears in any training subset we draw, exactly or after normalisation, at any training size or seed. The check is on the subsets rather than on the pool because the pool itself is not clean: it is what turns up the 3 rows above.

IV. Output Format Ladder

Comparing unrelated formats confounds length with surface form. We instead built six encodings of the *identical* tree that round trip to byte identical targets, four forming a length ladder and two held at matched length.

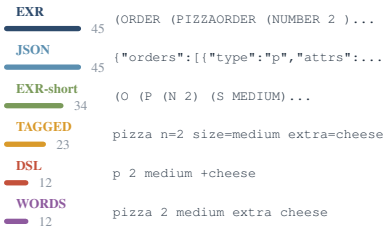


Fig. 2: The six encodings, all lossless and all preserving the dataset's own sibling order. EXR/JSON and DSL/WORDS are matched token pairs: nearly equal length, different surface form.

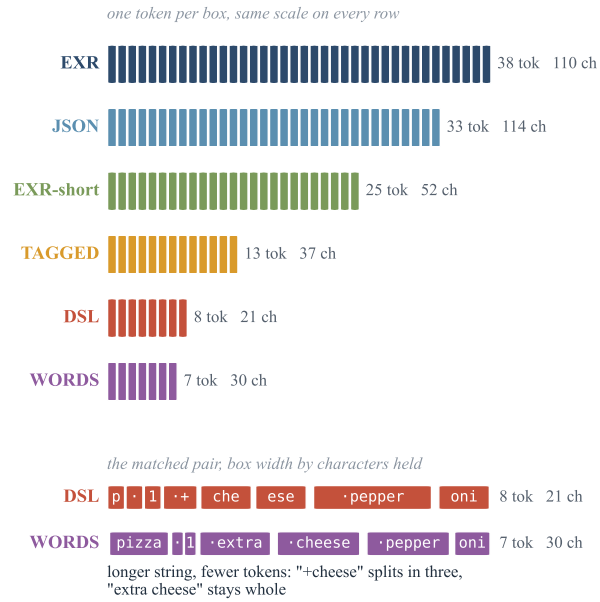


Fig. 3: The same order in all six encodings. Top: one box per token, all rows to one scale. Bottom: the matched pair with each box as wide as the characters it holds. WORDS is 30 characters against DSL's 21, nearly half as long again, and still costs fewer tokens (7 against 8), because `+cheese` splits into three subwords while `extra cheese` stays whole. Boundaries are read from the tokenizer's own merge table, not drawn.

EXR is the dataset's native s-expression. JSON is a conventional structured encoding at nearly identical token count. EXR-SHORT replaces constructor names with single letters. TAGGED is flat `key=value` pairs. DSL is a compact grammar with sigils, where `+` marks an extra quantity and `!` a negation. WORDS shares the DSL's structure but spells markers out as ordinary words.

All six decode back to the identical tree on every development and test utterance and on a 50,000 example training sample, checked against the dataset's own matcher.

a) A tokenizer floor.: Characters are not tokens, and the gap between them is where the ladder bottoms out. Figure 3 shows one real test utterance, *"i want a pie with pepperoni and extra cheese"*, in all six encodings, with every box a subword produced by the model's own vocabulary and merge table.

The bottom row is the floor in miniature. DSL spends three tokens on `+cheese`, since `+` is isolated and `cheese` after a sigil is split, where WORDS spends two on `extra cheese`. The symbolic form is shorter as text and longer as tokens. We also built a seventh encoding using short opaque entity codes, for example `p !p5 2 m3 +c2`, which takes the same effect to its limit: 16.4 characters per utterance against the DSL's 34.9, less than half, but *more* tokens, 11.9 against 10.8. Shortening the string stops reducing tokens well before the information content does, which is why no arm sits below roughly ten tokens and why we did not pursue that arm.

The floor is a property of the subword vocabulary [10], [11], not of the format, and the second model measures that directly. Moving from Qwen3.5-0.8B's 248,320 subwords to LFM2.5-350M's 65,536 costs every arm more tokens, but not equally: JSON inflates by 1.42× and EXR by 1.25×, while DSL inflates by 1.06× and WORDS by 1.14×. Punctuation-heavy structure is what a small vocabulary punishes; ordinary words are what it already contains.

V. Experimental Setup

Identical across all arms: LoRA [9] with r=16, α=16, batch 16, cosine schedule, loss on the completion only. Learning rate is selected per arm from $\{1e{-}4, 2e{-}4, 5e{-}4\}$ on development exact match at n=100,000 across three seeds, then frozen.

The primary model is Qwen3.5-0.8B [12], with adapters on all attention and MLP projections and the same prompt template the café model uses. The replication model is LFM2.5-350M [13]; Section VI-C reports it. Three things about it necessarily differ and we state them rather than implying a matched setup: ten of its sixteen blocks are convolutional, so there is no equivalent of an explicit attention projection list and we let the adapter library target every linear layer, which also reaches the convolution blocks; its prompt comes from its own chat template, which does not prefill a closed reasoning block as the Qwen template does; and its token caps are computed separately, because a cap is a property of the tokenizer. Each model has its own learning-rate sweep, its own caps and its own noise floor, and the two never share a run directory. Training uses early stopping on development exact match with patience 10, evaluated every 100 steps, to a maximum of 4000 steps.

`max_new_tokens` is set per arm to the longest target across all splits times 1.25, so no gold length output can be truncated. We log the share of generations reaching that cap; it is zero for every reported run.

Development drives every decision. Test is evaluated once per run and never read by any selection step. Exact match always comes from the dataset's own unordered matcher, which we never reimplement.

a) Metrics.: Write the test utterances $u_1 \dots u_n$ with gold trees g_i, and let o_i be the model's output and π the dataset's parser, with $\pi(o) = \bot$ when o is not well formed. Let $\simeq$ be the dataset's own unordered tree match. We report

$$V = \tfrac{1}{n}\left|\{\, i : \pi(o_i) \neq \bot \,\}\right| \qquad \text{(parse validity)}$$
$$A = \tfrac{1}{n}\left|\{\, i : \pi(o_i) \simeq g_i \,\}\right| \qquad \text{(exact match)}$$

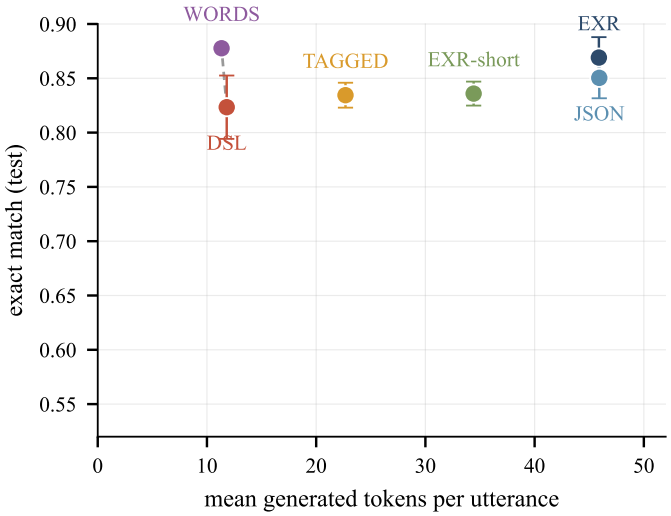


Fig. 4: Accuracy against output length. Dashed lines join the matched token pairs: within each pair length is held nearly constant, yet accuracy differs. Length alone does not determine accuracy.

and split the failures into the two kinds the introduction distinguishes:

$$\underbrace{A}_{\text{correct}} + \underbrace{S}_{\text{silent}} + \underbrace{D}_{\text{detectable}} = 1, \qquad S = V - A, \quad D = 1 - V.$$

S is the *silent error rate*: output a caller parses successfully and acts on, that is wrong. D is the share a caller can reject on sight. Figure 5 plots this decomposition for every arm. Note that A alone orders the arms differently from S, which is the practical point: an arm can win on validity and lose on the errors that actually reach a customer.

b) Noise floor.: We retrained one configuration twice under identical settings, per model. Test exact match differed by 0.032 on Qwen3.5-0.8B and 0.046 on LFM2.5-350M. Any difference smaller than the relevant floor is indistinguishable from rerun variance, and we do not interpret such differences.

VI. Results

A. Shorter is not automatically better

Table I and Figure 4 show no monotone benefit from shortening. Regressing exact match on mean generated tokens across the four ladder arms gives a *positive* slope at both training sizes: at n=1000, removing ten tokens costs about 1.5 points of exact match. Compression costs accuracy at both ends of the ladder. EXR-SHORT, which only renames the native format's constructors, gives up 0.033 exact match against EXR for a quarter of its length, and the worst arm overall is DSL at 0.823, one of the two shortest. Our original hypothesis, stated as "fewer output tokens is better," is falsified.

B. Surface form, not length

The matched token pairs separate the two explanations, under a paired bootstrap over the 1357 test utterances at every pool size and seed. The two pairs do not behave alike, and the difference between them is the result.

DSL and WORDS differ by less than a token in length, yet WORDS is ahead in all 15 of 15 cells and separates from the noise in 15 of them. The two encode the same structure; they differ only in whether a quantity marker is written `+cheese` or `extra cheese`. Symbolic surface forms cost accuracy that natural word forms do not.

The EXR/JSON pair is the weaker half of the evidence and we report it as such. EXR is ahead in 13 of 15 cells, so the direction is consistent, but only 8 of them clear the noise floor. That is the expected pattern rather than a disappointment: both members of this pair are punctuation-heavy symbolic notations, differing in which brackets they use, so there is little of the natural-word axis between them to measure. The pair that varies that axis is the pair that separates everywhere.

C. Replication on a second model

The result above is the one that carries the argument, so it is the one worth trying to break. The obvious way it could be an artifact is the tokenizer: if WORDS wins because the Qwen vocabulary happens to contain the words this task uses, a model built on a different vocabulary should behave differently. We therefore repeated the whole grid on LFM2.5-350M, whose vocabulary is 65,536 against Qwen3.5-0.8B's 248,320, a factor of nearly four, and whose architecture is largely convolutional rather than attentional.

Three things survive the change of model. The best arm is WORDS again. The matched pair keeps both its sign and its size: WORDS over DSL by 0.054 on Qwen3.5-0.8B and 0.057 on LFM2.5-350M, against rerun noise floors of 0.032 and 0.046. And the ladder is non-monotone on both, so the length story fails in the same way twice.

The sign of the token difference is what makes this more than a repetition. Under Qwen3.5-0.8B, WORDS is marginally the shorter of the pair (11.3 against 11.8 tokens), so a sceptic can still attribute its advantage to length. Under LFM2.5-350M the smaller vocabulary fragments the word forms more and the ordering inverts: WORDS costs 12.9 tokens against DSL's 12.6, and it is *still* the more accurate of the two, by 0.057. A format that uses more tokens and makes fewer mistakes cannot be winning because it is short.

We draw no conclusion from the gap between the two models' absolute scores. They differ in parameter count, in architecture, in which modules carry adapters, and in whether the prompt prefills a closed reasoning block; any of those could account for it, and the design separates none of them. The replication is of the ordering, which is what the ladder and the matched pairs were built to isolate.

D. The café task does not resolve the question

The café parser was run as the same comparison on a different task, in Russian and English rather than English alone, and on the system in production. It has two arms rather than six, so it contributes no ladder and no slope: COMPACT is the shipped format and WORDS spells the same fields out, and they round-trip to byte-identical output. It did not replicate the format effect, and it could not have: the task is saturated.

TABLE I: Test results for Qwen3.5-0.8B at n=2,456,443, mean of three seeds. Silent error is the share of utterances that parse cleanly but do not match the gold tree: the caller cannot detect these. Latency is median at batch size one on a shared GPU, and memory is this process's own GPU allocation during that measurement.

Format	Tokens	Exact match	Parse validity	**Silent error**	Halluc./utt.	Latency (ms)	Mem. (MiB)
EXR	45.9	0.869 ± 0.019	0.993	0.124	0.113	1168	1793
JSON	45.9	0.850 ± 0.019	0.985	0.134	0.132	1203	1707
EXR-short	34.4	0.836 ± 0.011	0.970	0.134	0.139	901	1706
TAGGED	22.7	0.834 ± 0.011	0.989	0.154	0.155	622	1706
DSL	11.8	0.823 ± 0.029	0.937	0.113	0.094	321	1705
WORDS	11.3	0.878 ± 0.004	0.940	0.062	0.045	318	1705

TABLE II: Matched token pairs at n=2,456,443, the headline size only; the counts quoted in the text are over all 5 sizes. Paired bootstrap over test utterances. Equal length, different surface form.

Pair	Seed	Tokens	Δ EM	95% CI	Verdict
EXR/JSON	0	46.2/45.9	+0.013	[-0.005, +0.030]	noise
EXR/JSON	1	45.7/46.0	-0.015	[-0.032, +0.001]	noise
EXR/JSON	2	45.8/45.9	+0.058	[+0.041, +0.077]	sep.
DSL/WORDS	0	11.9/11.4	-0.091	[-0.111, -0.071]	sep.
DSL/WORDS	1	11.8/11.3	-0.034	[-0.052, -0.015]	sep.
DSL/WORDS	2	11.7/11.4	-0.038	[-0.057, -0.018]	sep.

TABLE III: Mean output tokens and test exact match at n=2,456,443, three seeds, best arm in bold. The two models are not comparable on absolute score and are not compared on it; what replicates is the ordering.

	Qwen3.5-0.8B		LFM2.5-350M	
	tok.	EM	tok.	EM
EXR	45.9	0.869	57.5	0.740
JSON	45.9	0.850	65.0	0.751
EXR-SHORT	34.4	0.836	39.7	0.718
TAGGED	22.7	0.834	26.3	0.759
DSL	11.8	0.823	12.6	0.748
WORDS	11.3	**0.878**	12.9	**0.805**

Every cell of the grid, across both arms and all four pool sizes, falls between 0.939 and 0.980 exact match. With 49 held-out cases one utterance is worth 2.0 points, so the instrument cannot see anything smaller than that, and the -0.007 between the two arms at the headline size is about a third of one case. All 12 matched pairs fall inside the paired bootstrap interval, and the determinism replicate puts the run-to-run floor at 0.020, which is one case exactly. The honest reading is that at this size the café set measures a resolution limit rather than a format difference, in either direction.

We report it rather than dropping it for two reasons. It is the negative result that bounds the claim: the effect the earlier sections measure is a few points, and a few points is invisible to a fifty case set, so anyone who evaluates a production parser on a hand-written suite of this size will see nothing and should not conclude there is nothing. And it does establish that the shipped format works, at 117 ms per order against 147 ms for the longer one, on the same task the device measurements in Section III-A were taken from. Settling the format question on café orders needs a larger and harder held-out set, drawn from real transcripts rather than written by hand; that set does not exist yet.

TABLE IV: Café task at n=100,000, three seeds, on the 49 case held-out set. No arm is marked best, because none separates.

arm	tokens	EM	validity
COMPACT	3.2	0.980	0.993
WORDS	4.7	0.973	0.993

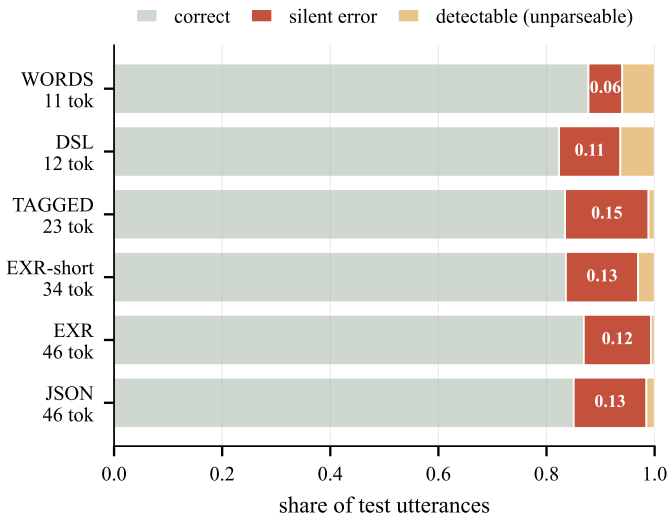


Fig. 5: Where the test set goes for each format, ordered by output length. Silent errors parse cleanly but are wrong, so a downstream system cannot detect them. Detectable failures can be rejected and retried.

E. Silent errors

Figure 5 is the deployment picture. EXR has the highest parse validity at 0.993, but that is not an unalloyed good: nearly everything it gets wrong, it gets wrong *silently*, at 0.124. WORDS parses less often, 0.940, but its silent rate is 0.062, a factor of 2.0 lower. Its errors surface as parse failures a caller can catch.

The hallucination counts move the same way: 0.045 invented entities per utterance against 0.113 for EXR, a factor of 2.5, and 0.056 missing entities against 0.168.

F. Latency

Latency is close to proportional to output length (Figure 6). WORDS responds in 318 ms against 1168 ms for EXR, a 3.7× speedup, on a shared GPU under contention from other

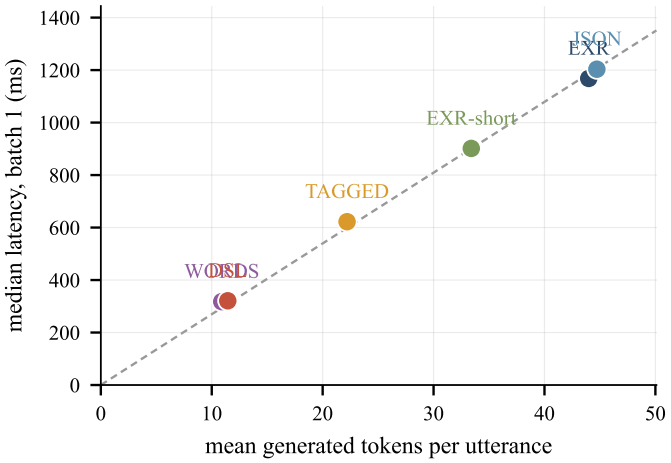


Fig. 6: Median latency at batch size one is close to linear in generated tokens. Throughput in tokens per second is flat across formats, so the speedup comes from emitting less, not from running faster.

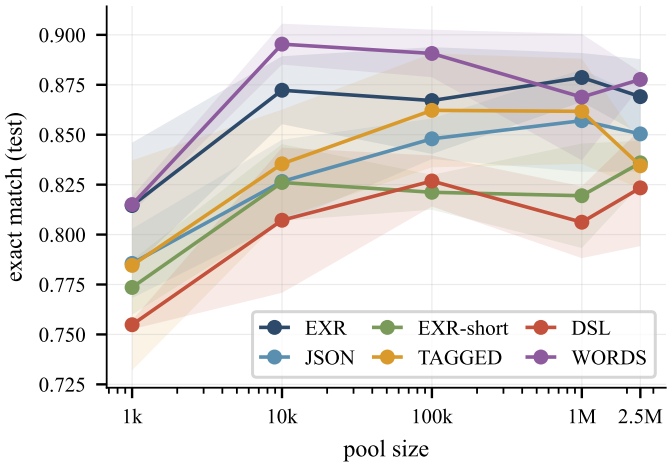


Fig. 7: Exact match against training set size. The penalty for symbolic formats is largest when data is scarce, and narrows as data grows.

tenants. Tokens per second is flat across arms, confirming the gain comes from length rather than from a faster model.

Memory tells a different story. Across all eighteen measurements the process holds between 1705 and 1793 MiB, a spread of about fifteen percent that tracks the longer key value cache the verbose formats need. Weights dominate, and those are identical across arms. **Format choice buys latency and accuracy, not memory.** A practitioner picking a format to fit a memory budget is picking the wrong lever; the lever for memory is quantisation, which is what the deployed client in Section VI-H uses.

G. Data efficiency

Figure 7 shows the format penalty is a low data phenomenon. At n=1000, DSL reaches 0.755 against WORDS at 0.815. By n=2,456,443 the gap narrows. A format closer to natural language appears to let the model reuse what pretraining already gave it, which matters most when there is little task data to learn from.

H. Deployment on device

The café parser runs entirely on an iPad, with no network call. The client is a SwiftUI application that links llama.cpp as an `xcframework` and offloads every layer to the Metal GPU. Table V lists what it costs to carry.

TABLE V: On-device footprint, measured on an iPad (A16) by the client's own benchmark: ten representative orders, three timed rounds after a discarded warm-up pass. The model is the café parser quantised to Q8_0.

Device	iPad15,7, iOS 26.3
Base model	Qwen3.5, 0.773 B parameters, 25 blocks
Quantisation	Q8_0, 8.63 bits per weight
Model file	795 MiB
Inference runtime	llama.cpp, 11 MiB framework (arm64)
GPU offload	all layers, Metal
Context window	512 tokens
Peak resident footprint	350 MiB
Model load time	2.5 s
Median latency	167 ms
p90 latency	354 ms
Throughput	16 tok/s

The spread between the median and the p90 is itself the paper's argument, restated on the hardware a customer waits in front of. Across the ten orders the benchmark parses, latency is close to linear in the number of output tokens: a least squares fit gives 48 ms of fixed overhead plus 52 ms per generated token ($R^2 = 0.91$), so a one token reply such as `esp` returns in about 100 ms while the longest, a seven token two drink order, takes about 414 ms. The format's token count is therefore not a throughput abstraction but the dominant term in the wait, and the 4.0× reduction the word based encoding buys becomes the same factor of responsiveness at the booth.

The footprint deserves a note, because 350 MiB is well under the 795 MiB model file. The figure is `phys_footprint`, which is what iOS enforces its per-process memory limit against, and llama.cpp maps the weight file rather than copying it, so the weight pages are file backed and are reclaimable under pressure instead of counting against that limit. The number to plan a deployment around is therefore the resident figure, not the file size, and on this device it leaves the parser a long way inside the budget of even an entry level iPad.

Two further details matter for reproducibility. First, the prompt template and the three stop strings compiled into the client are byte identical to those used for training and for every evaluation in this paper, so the deployed model sees exactly the input distribution it was fitted on. Second, the context window is 512 tokens, which the compact format fits comfortably and the equivalent JSON encoding would not at the longest orders. A format chosen for token count at training time therefore also decides whether the model fits a small on-device context at serving time.

The practical consequence is that a booth needs no server, no network round trip, and no per-request cost. The whole parser is a 795 MiB file beside an 11 MiB runtime.

I. Reproducibility

Every number in this paper is recomputed from saved model outputs rather than copied from a training log. Each model has its own grid of 90 runs under its own directory, so no token cap, learning rate or noise floor is ever shared between them. Each run writes its configuration, package versions, training curves, adapter weights and the full prediction file for both evaluation splits, together with the generation contract it used: padding side, stop token and id, vocabulary size and the resolved prompt. We record the last of these because an earlier replication grid was run with right padding, which places pad tokens between the prompt and the continuation, and scored between 0.13 and 0.20 lower on every arm; those runs were discarded and rerun, and the arm ordering was unaffected. A prediction row carries the source utterance, the gold tree, the raw model output, the parsed tree or a null, a correctness flag and the generated token count, so exact match can be recomputed offline with no GPU. Every table and figure here is produced by two scripts that read only those files. The protocol, including the stopping rules and the statements this work declines to make, was written before any run and records its own later amendments.

VII. Discussion

a) Which format should a practitioner choose?: WORDS. On Qwen3.5-0.8B it ties the native format on accuracy within the noise floor and wins on every deployment axis: 4.0× fewer tokens, 3.7× lower latency, 2.5× fewer hallucinations, and 2.0× fewer silent errors. It is also the best arm on LFM2.5-350M, by a margin outside that model's own noise floor, so the recommendation does not rest on one tokenizer. The recipe is to compress the *structure* while keeping the *vocabulary* natural.

b) How small can the model be?: At 2,456,443 training examples Qwen3.5-0.8B reaches 0.878 exact match on WORDS. LFM2.5-350M, less than half the parameters, reaches 0.805 on the same arm.

This is now a like-for-like comparison on data, which earlier versions of this study could not make. The published BART-Large EXR result on this benchmark is 0.786, trained on 2,456,446 utterances; our headline grid trains on 2,456,443, the same pool less the 3 rows that duplicate an evaluation utterance and that we drop as contamination. At that budget Qwen3.5-0.8B is 0.092 ahead on its best arm. We still decline a state-of-the-art claim, but the reason is now narrower than it was: not the data budget, which is matched, but the architecture and training regime. BART-Large EXR is a fully fine tuned encoder-decoder; this is a decoder-only model of a different size carrying LoRA adapters, and neither design was tuned to compete with the other. The number is a reference line, and what we read off it is only that a sub-billion-parameter model with adapters is in the same range as the published baseline once the format is chosen well.

What is worth noting is the size of the format effect against the size of the model effect. Choosing the format moves exact match by 0.054 across the six arms on Qwen3.5-0.8B and 0.086 on LFM2.5-350M, while the two models differ by 0.073 on their best arm. On the smaller model the format matters more than the choice of model; on the larger one the two effects are comparable. Either way, format is not a rounding error next to capacity. Both sizes serve in real time on consumer hardware, which the café system in Section III-A does at 80 ms per order.

c) Limitations.: One task that resolves the question and two models, neither of them large. The replication in Section VI-C shows the ordering survives a fourfold change of vocabulary, but both models are under a billion parameters and both grids are the same 1,357 utterance test split, so the finding is not established beyond this task. The second task we ran, in Section VI-D, was saturated and separates nothing in either direction; it bounds the claim rather than supporting it. The replication was also added after the primary grid rather than pre-registered, and is reported as such. Three seeds per cell gives wide intervals, which is why arm comparisons use a paired bootstrap over utterances rather than seed means. Learning rate is tuned at $n{=}100{,}000$ and reused at every pool size. Latency is measured on a contended shared GPU, so absolute milliseconds are inflated, though every arm meets the same contention. We have not tested whether these findings transfer to a larger model, where the format penalty may vanish. Finally, the two models differ in capacity, architecture, adapter placement and prompt conditioning at once, so nothing here attributes any difference between their absolute scores to any one of those.

VIII. Conclusion

Output format is a free design variable that changes hallucination rate and latency by factors of three to four. The intuition that fewer tokens is safer is not quite right. What helps is compressing structure while keeping the surface in natural words, and what matters for deployment is not overall accuracy but how often a model is wrong in a way nobody can detect. The ordering survives a fourfold change of subword vocabulary, and on the smaller vocabulary the word based format costs more tokens than its symbolic twin while remaining more accurate, which is the clearest evidence we have that the surface form and not the length is doing the work.

References

[1] K. Arkoudas *et al.*, "PIZZA: A new benchmark for complex end-to-end task-oriented parsing," 2022. Dataset: https://github.com/amazon-science/pizza-semantic-parsing-dataset

[2] L. Dong and M. Lapata, "Language to logical form with neural attention," in *Proc. ACL*, 2016, pp. 33–43.

[3] R. Shin *et al.*, "Constrained language models yield few-shot semantic parsers," in *Proc. EMNLP*, 2021, pp. 7699–7715.

[4] Z. R. Tam *et al.*, "Let me speak freely? A study on the impact of format restrictions on performance of large language models," in *Proc. EMNLP Industry Track*, 2024, pp. 1218–1236.

[5] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proc. EMNLP*, 2021, pp. 9895–9901.

[6] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proc. EMNLP*, 2023, pp. 10932–10952.

[7] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv:2307.09702, 2023.

[8] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, art. 248, 2023.

[9] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.
[10] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. ACL*, 2016, pp. 1715–1725.
[11] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? On the monolingual performance of multilingual language models," in *Proc. ACL-IJCNLP*, 2021, pp. 3118–3135.
[12] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.
[13] A. Amini *et al.*, "LFM2 technical report," arXiv:2511.23404, 2025.